Bipedal Walker

Reinforcement Learning

Francisca Cerqueira (nº 202303764)
Iara Ferreira (nº 202305194)
Rodrigo Simões (nº 202303448)

Formalização do Problema

O problema é formulado como um **Markov Decision Process (MDP)** totalmente observável. O ambiente fornece um vetor de observação contínuo $S \subset IR^{24}$, que inclui:

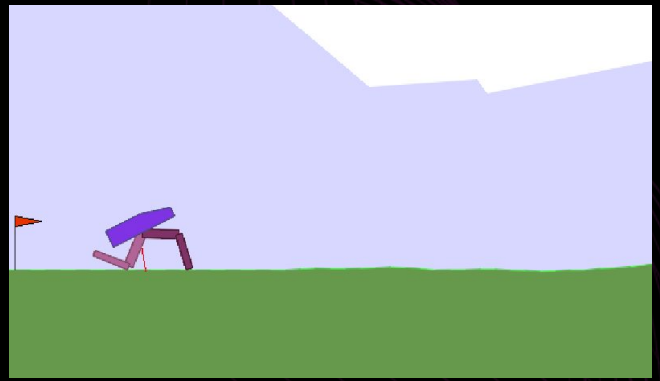
- **Propriocepção:** Ângulos e velocidades angulares do casco e das articulações. (hull_angularVelocity , hip_joint_1_angle, hip_joint_1_speed, knee_joint_1_angle, knee_joint_1_speed, hip_joint_2_angle, hip_joint_2_speed, knee_joint_2_angle e knee_joint_2_speed)
- **Dinâmica:** Velocidades lineares. (vel_x e vel_Y)
- **Sensores de Terreno:** 10 leituras de LIDAR que medem a distância ao solo e irregularidades. (10 lidar readings)
- **Contacto:** Variáveis booleanas que indicam se as pernas tocam no solo. (leg_1_ground_contact_flag e leg_2_ground_contact_flag)

	Num	Observation	Min	Max
0	0	hull_angle	-3.141593	3.141593
1	1	hull_angularVelocity	-5.000000	5.000000
2	2	vel_x	-5.000000	5.000000
3	3	vel_y	-5.000000	5.000000
4	4	hip_1_angle	-3.141593	3.141593
5	5	hip_1_speed	-5.000000	5.000000
6	6	knee_1_angle	-3.141593	3.141593
7	7	knee_1_speed	-5.000000	5.000000
8	8	leg_1_ground_contact_flag	-0.000000	5.000000
9	9	hip_2_angle	-3.141593	3.141593
10	10	hip_2_speed	-5.000000	5.000000
11	11	knee_2_angle	-3.141593	3.141593
12	12	knee_2_speed	-5.000000	5.000000
13	13	leg_2_ground_contact_flag	-0.000000	5.000000
14	14	lidar_0	-1.000000	1.000000
15	15	lidar_1	-1.000000	1.000000
16	16	lidar_2	-1.000000	1.000000
17	17	lidar_3	-1.000000	1.000000
18	18	lidar_4	-1.000000	1.000000
19	19	lidar_5	-1.000000	1.000000
20	20	lidar_6	-1.000000	1.000000
21	21	lidar_7	-1.000000	1.000000
22	22	lidar_8	-1.000000	1.000000
23	23	lidar_9	-1.000000	1.000000

Descrição do Ambiente

	Num	Name	Min	Max
0	0	Hip_1 (Torque/Velocity)	-1.0	1.0
1	1	Knee_1 (Torque/Velocity)	-1.0	1.0
2	2	Hip_2 (Torque/Velocity)	-1.0	1.0
3	3	Knee_2 (Torque/Velocity)	-1.0	1.0

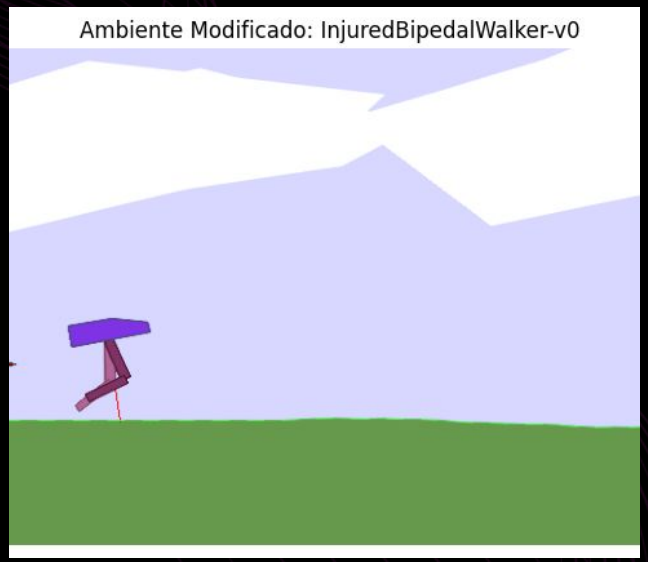
O agente atua num **espaço contínuo quadridimensional**: $A = [-1, 1]^4$. Cada dimensão controla o torque aplicado a uma junta específica. Os valores são **normalizados** entre -1 e 1 e convertidos em força pelo motor de física Box2D.



A **função de recompensa** original incentiva o progresso e a eficiência:

- **(+) Avanço:** Recompensa proporcional à distância percorrida em frente.
- **(-) Custo Energético:** Penalização proporcional ao torque aplicado (incentiva movimentos suaves).
- **(-) Falha:** Penalização severa (-100) em caso de queda do robô.

Mudança Introduzida
InjuredBipedalWalker-v0



Simulamos uma perda de potência de **70% na perna direita**. Isto é implementado interceptando o vetor de ação antes de este chegar ao motor de física:

```
class InjuredBipedalWalker(BipedalWalker):
    def step(self, action):
        # Índices 2 e 3 correspondem à Anca e
Joelho da perna direita
        action[2] *= 0.3  # Redução de força
        action[3] *= 0.3
        return super().step(action)
```

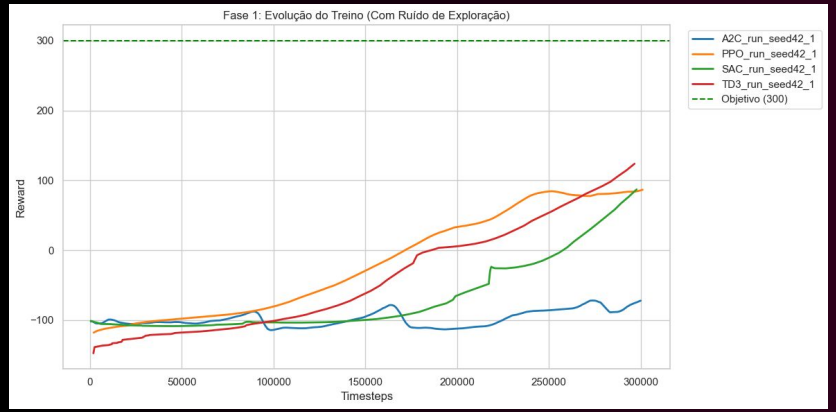
Algoritmos Considerados

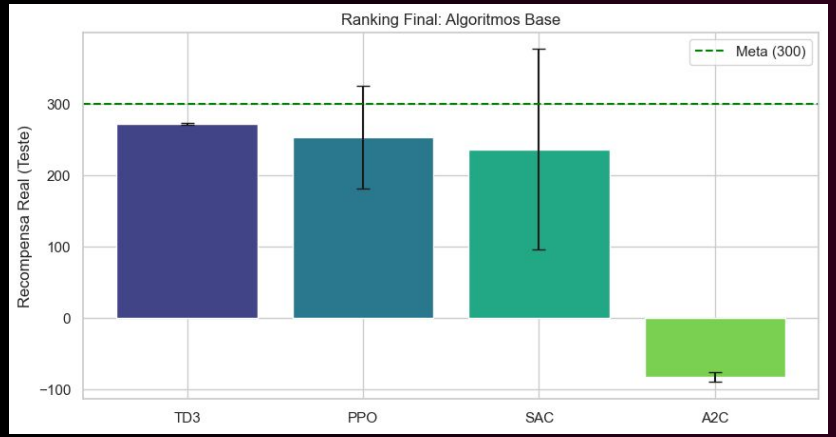
Submetemos quatro algoritmos a um treino de **300.000 passos:**

Os algoritmos testados:

1. **PPO & A2C (On-Policy):** Geralmente mais estáveis, mas menos eficientes no uso de dados (sample efficiency).
2. **SAC & TD3 (Off-Policy):** Geralmente mais instáveis, mas capazes de aprender muito mais rápido em robótica contínua.

Algoritmo que mostrou melhor desempenho global: **TD3**.



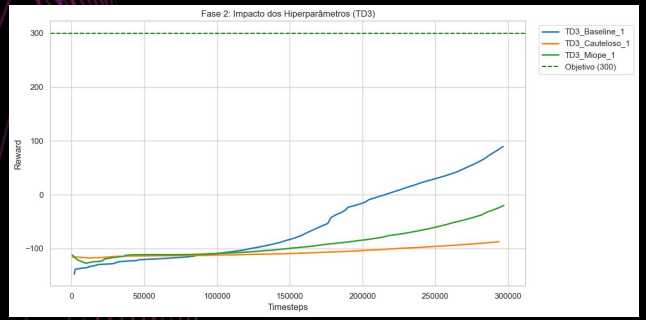


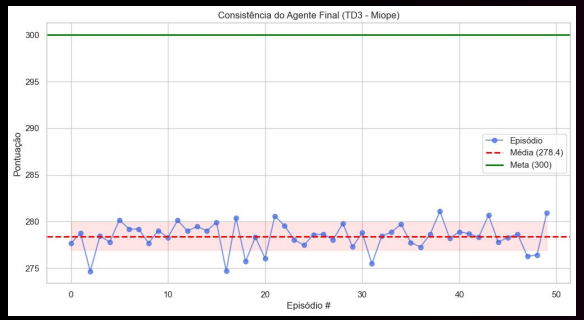
Otimização de Hiperparâmetros

No Ambiente Original

Nesta fase, testamos três variantes do TD3 com **300.000 passos** e Seed fixa:

1. **Baseline (LR=0.001):** A configuração padrão. É agressiva e pode ser instável.
2. **Cauteloso (LR=0.0003):** Uma taxa de aprendizagem menor. Espera-se uma convergência mais lenta, mas mais sólida e com pontuação final mais alta.
3. **Míope (Gamma=0.98):** Reduzimos o horizonte de planeamento para focar o agente no equilíbrio imediato.



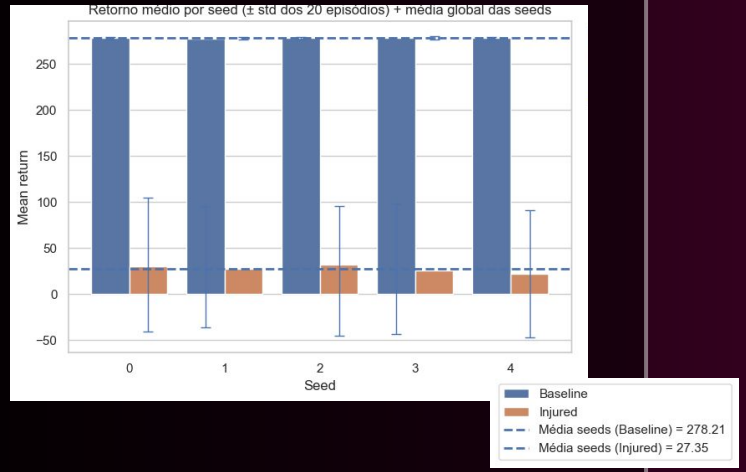


Config	Média	Std	Path
2	Miope	278.505737	1.695485
0	Baseline	254.212311	76.888695
1	Cauteloso	-58.019966	15.599453

Avaliação Determinística Cruzada
Baseline VS Modificado

A primeira experiência que realizamos foi a avaliação d**o mesmo modelo** (TD3-Míope treinado no `BipedalWalker-v3`) em dois contextos:

1. **Ambiente baseline** (`BipedalWalker-v3`) — referência "saudável".
2. **Ambiente modificado** (`InjuredBipedalWalker-v0`) — cenário problemático (perda de força numa perna).



Implicação: para obter um desempenho representativo no cenário modificado e um *upper bound* credível, é necessário **treinar diretamente** no ambiente modificado e, potencialmente, ajustar hiperparâmetros que controlam exploração e estabilidade.

Treino do Agente no Ambiente Modificado

Variamos hiperparâmetros com impacto direto em adaptação no controlo contínuo:

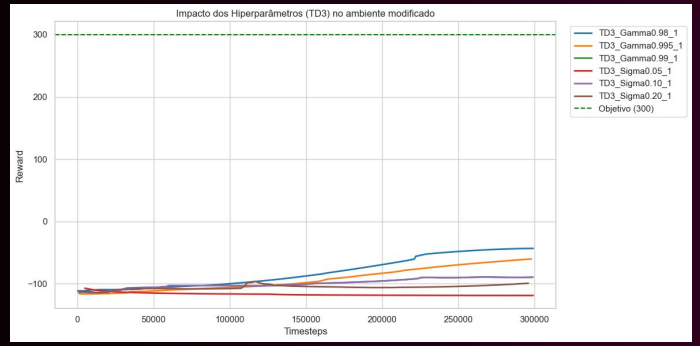
Exploração (ruído de ação, σ): {0.05, 0.10, 0.20}

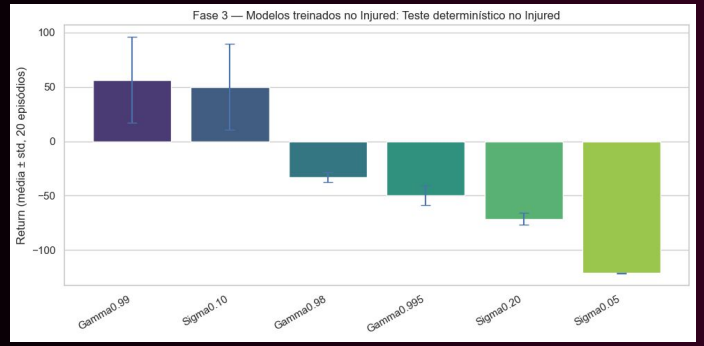
Horizonte (γ): {0.98, 0.99, 0.995}

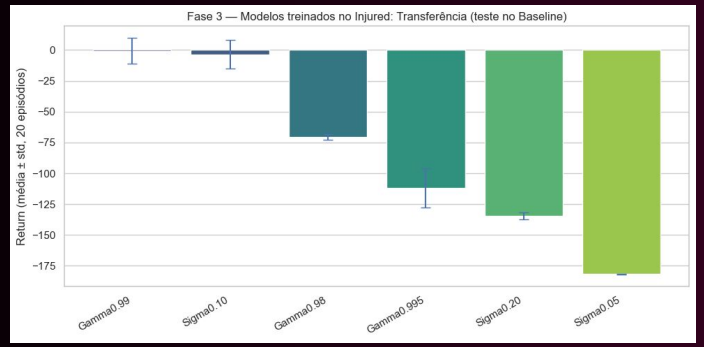
Sequindo o critério de **maximizar retorno no Injured**, as melhores escolhas são **Gamma0.99** ou **Sigma0.10**, mas com o alerta de que são **muito instáveis** (Std ≈ 39).

Seguindo o critério de **consistência**, **Gamma0.98** é mais estável, mas não atinge desempenho positivo no Injured.

Em qualquer caso, a **transferência inversa (Injured → Baseline)** é fraca, sugerindo que a adaptação à lesão produz uma política especializada (possivelmente assimétrica).



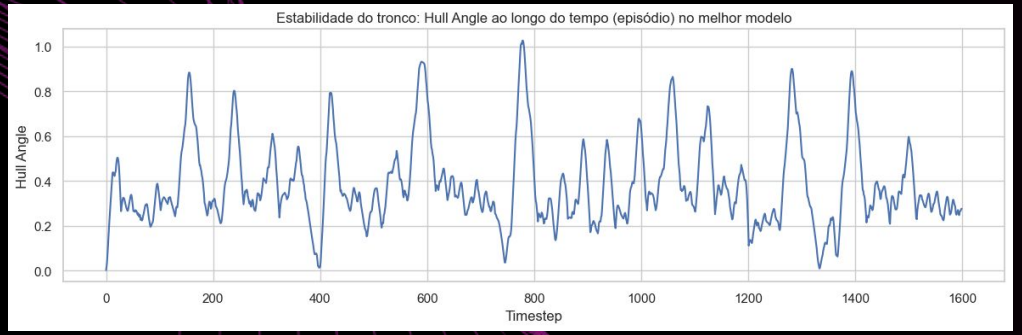


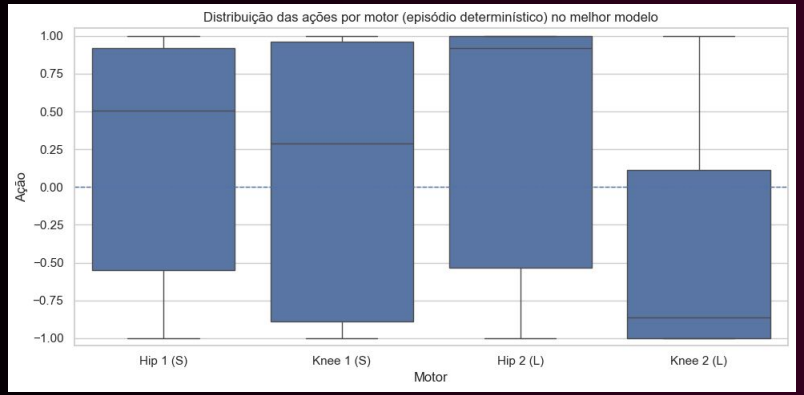


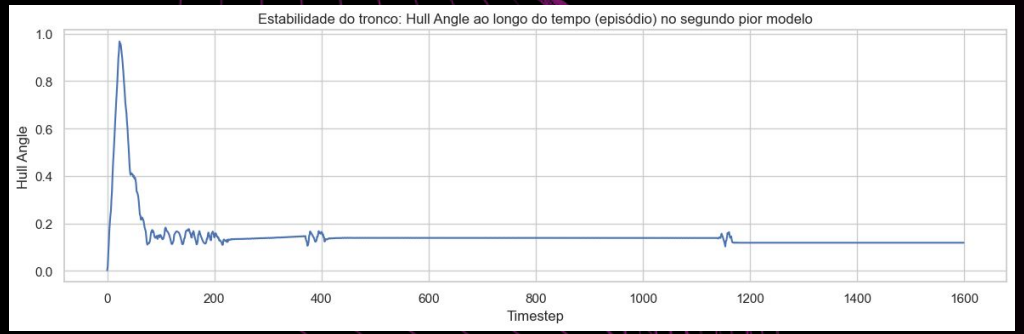
Análise Comportamental

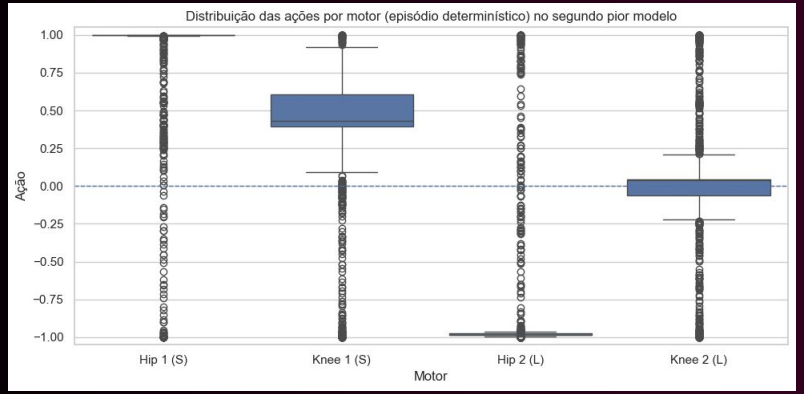
Timeout:	96%
Queda:	4%

1. Distribuição das ações por motor
2. Estabilidade do Tronco (Hull Angle)
3. Análise de falhas: como o agente perde?









Conclusões

- **Robustez ≠ desempenho no Baseline:** uma política excelente no ambiente saudável pode falhar quase completamente sob falha mecânica.
- **Treinar no Injured muda a estratégia:**
  - ações tornam-se **mais assimétricas**
  - tronco mostra maior oscilação ⇒ marcha **compensatória**
- **Falha dominante:** no melhor agente lesionado, a maioria dos episódios termina por **timeout**, o que indica que o agente aprendeu sobretudo a **sobreviver** (não cair), mas ainda não a **progredir com eficiência**.

Future Work

Para tornar o agente simultaneamente **adaptável** e **generalizável**, seria natural explorar:

- **Curriculum learning:** aumentar gradualmente a severidade do dano (0.9 → 0.3)
- **Domain randomization:** variar dano/terreno durante treino para forçar robustez
- **Estratégias robustas:**regularização, treino multi-task, ou políticas condicionadas no “nível de dano”